

BREAKPOINT V5

Rapport technique officiel — finalisation gameplay, modèle économique et gel pré-AAB

Étude, audit et préparation d'état uniquement. Aucun SDK publicitaire réel (AdMob), aucun Google Play Billing réel, aucune fausse publicité et aucun faux écran d'achat n'ont été intégrés dans cette PWA. Le système de vies n'a pas été modifié (décision fondée sur les données, voir section 3). Le gameplay, les contrôles tactiles, les collisions, les bonus, l'audio et la progression déjà validés sont intacts.

Dépôt : `scuillerjack-oss/breakpoint-v0`, branche `main`. Date : 25 septembre 2026.

1. État Git — audit préalable et état final

Élément	Valeur
HEAD au début de la mission V5	7e5743e ("BREAKPOINT V4 : étude et modèle économique chiffré")
git status au début	propre, distant identique au local (vérifié par <code>git ls-remote</code>)
Commit de code V5	c709fac — audit des vies, étude de simulation, modèle économique, préparation <code>monetization.js</code>
Commit correctif	ab55570 — corrige un flake réel détecté par le run CI réel du commit c709fac (test de terminabilité des niveaux, RNG des power-ups non semé — voir section 6)
HEAD final (vérifié)	ab55570a36e66e67ccc3b19b9127807ff7bc7a58
Remote après push	identique au local, confirmé par <code>git ls-remote</code> (pas une supposition)
Déploiement	run CI #13 (id 36127927933) — jobs build ET deploy tous deux "success", vérifié au niveau des jobs, pas seulement le statut global du workflow
Identifiant de build servi	v0.1.0+ab55570 (reconstruit localement avec <code>GITHUB_SHA=ab55570...</code> , identique à ce que le build CI a produit)

Incident réel rencontré et corrigé en cours de mission : le premier push (commit c709fac) a échoué en CI réelle (job build, étape `npm test`) sur `tests/levels-reachability.test.js`, niveau 13 — un flake préexistant (non introduit par cette mission) causé par le tirage aléatoire réel des power-ups (`Math.random()` non semé dans `applyPowerUp()`), sous un tirage suffisamment défavorable. Diagnostiqué à la cause (pas juste re-exécuté en espérant que ça passe), reproduit localement (1 échec sur 5 exécutions), corrigé en semant `Math.random` avec un générateur à graine fixe DANS LE TEST uniquement (le moteur de jeu réel garde son vrai aléa en jeu normal). Non-vacuité vérifiée : 8/8 exécutions propres après correctif. Second push (ab55570) confirmé vert en CI réelle.

2. Audit obligatoire du système de vies (avant toute modification)

Effectué directement dans le code réellement servi (aucune supposition) : `src/engine/state.js`, `simulation.js`, `src/main.js`.

Événement	Comportement réel constaté
Portée des 3 vies	<code>state.lives</code> est un champ de l'état de NIVEAU (<code>createLevelState</code>), initialisé à <code>carry.lives</code> ?? 3. Dans l'app réelle, <code>main.js</code> appelle TOUJOURS <code>createLevelState(index, {lives: runLives, score: runScore})</code> — jamais sans <code>carry</code> .
Changement de niveau (niveau réussi)	<code>startLevel(levelIndex+1)</code> est appelé avec <code>runLives</code> INCHANGÉ depuis la fin du niveau précédent — les vies sont donc CONSERVÉES, pas réinitialisées, entre deux niveaux réussis consécutifs.
Perte de balle/vie	<code>handleLifeLost()</code> décrémente <code>state.lives</code> ; si >0 , une balle neuve réapparaît (status "ready") ; si ≤ 0 , status devient "lost" (Game Over).
Game Over → "Recommencer"	<code>runLives</code> et <code>runScore</code> sont remis à 3/0 AVANT de rejouer le MÊME niveau (<code>showLevelResult(false)</code> , bouton <code>btn-retry</code>).
Game Over → "Menu"	<code>runLives</code> et <code>runScore</code> également remis à 3/0 avant <code>showMenu()</code> .
Pause → "Recommencer" (abandon volontaire, PAS un Game Over)	<code>confirmAbandon()</code> => <code>startLevel(levelIndex)</code> — <code>runLives</code> N'EST PAS réinitialisé : le niveau est rejoué avec le nombre de vies qu'il restait au moment de la pause. Comportement réel, asymétrique avec le cas Game Over, mais PAS un bug de réinitialisation involontaire au sens du cahier des charges (aucune vie n'est perdue ou gagnée sans raison) — documenté tel quel, non modifié (hors périmètre de la décision fondée sur les données, section 3).
Pause → "Retour au menu" (abandon volontaire)	<code>confirmAbandon(showMenu)</code> — <code>runLives</code> N'EST PAS non plus réinitialisé ici. Un "Continuer" ultérieur depuis le menu réutilise cette valeur. Documenté, non modifié pour la même raison.
Sauvegarde/reprise (fermeture complète de l'app)	<code>save.js</code> ne persiste JAMAIS les vies/score en cours — seul <code>unlockedLevelIndex</code> (progression de niveau débloqué) survit. Un rechargement complet réinitialise TOUJOURS <code>runLives</code> à 3 (valeur littérale du module au chargement).

Conclusion de l'audit : aucun comportement observé ne provient d'un bug ou d'une réinitialisation involontaire au sens strict (aucune vie n'apparaît ou ne disparaît sans code explicite qui l'explique). Le seul point notable est l'asymétrie pause-abandon vs Game Over ci-dessus, documentée mais non corrigée car hors du périmètre de la question posée par le cahier des charges (le système de VIES ENTRE NIVEAUX, pas les transitions de menu).

3. Étude de simulation des vies — Scénarios A / B / C

3.1. Méthodologie

- Rejoue une campagne COMPLÈTE des 50 niveaux réels dans l'ordre, exactement comme un joueur qui enchaîne "Niveau suivant" — pas des niveaux isolés.
- 5 profils de joueur déterministes, du plus précis (2 profils déjà utilisés par tests/levels-reachability.test.js depuis V1, mesurés empiriquement : 0 Game Over sur les 50 niveaux) au moins précis (délai de réaction simulé — la raquette suit une position RETARDÉE de la balle — et fenêtres de "décrochage"/distraction périodiques pour les profils "Faible" et "Débutant"). Aucun niveau n'est modifié : seul le modèle du JOUEUR varie.
- 8 graines indépendantes par combinaison (joueur × politique) : le tirage réel des power-ups (Math.random non semé dans le moteur) est contrôlé par un générateur à graine fixe LE TEMPS DE L'ÉTUDE, pour obtenir une distribution plutôt qu'un tirage chanceux ou malchanceux isolé.
- Sur un Game Over, la simulation modélise le comportement réel observé (bouton "Recommencer" : vies remises à 3, même niveau rejoué) puis continue — jamais un arrêt, pour mesurer la fréquence réelle sur toute la campagne.

3.2. Résultats — moyenne sur 8 graines, Game Over par campagne complète (50 niveaux)

Profil de joueur	Politique A (actuelle) — moyenne GO	min-max (8 graines)	Politique B (reset/niveau) — moyenne GO
Expert	0	0-0	0
Bon	0	0-0	0
Moyen	0	0-0	0
Faible	12.13	10-14	0
Debutant	15.75	14-17	0

Politique C (restauration par palier, testée à N=10/20/30 niveaux) : total moyen de Game Over — N=10→22.63, N=20→25.76, N=30→27 — à comparer à A=27.88. Réduction de seulement 3 à 19% par rapport à A, sans amélioration qualitative claire (les profils déjà à l'aise restent à 0 Game Over sous A comme sous C ; seuls les profils "Faible"/"Débutant" voient une réduction modeste).

3.3. Décision fondée sur les données

Politique A (comportement actuel) est CONSERVÉE — aucune modification du système de vies.

- Politique B élimine ENTIÈREMENT tout Game Over, y compris pour les profils les moins précis simulés — ce qui rendrait le rewarded "continuer" économiquement INSIGNIFIANT (jamais déclenché en pratique), exactement le risque explicitement écarté par le cahier des charges (section 3.2 : "éviter... une mécanique économiquement insignifiante").
- Politique A laisse les profils à l'aise (3 des 5 profils simulés) totalement sans Game Over — cohérent avec le retour de bêta positif déjà obtenu sur le gameplay actuel — tout en laissant une minorité réaliste de joueurs moins précis rencontrer un vrai Game Over, préservant une fréquence rewarded réelle et non nulle.
- Politique C n'apporte qu'un compromis marginal (3-19% de réduction) sans justification claire d'un changement, pour une complexité additionnelle réelle (nouvel état persistant, nouveaux cas de test) — non retenue.
- Ce résultat est explicitement autorisé par le cahier des charges V5 (section 9) : "si les résultats confirment les choix" (ici : confirment que le comportement actuel reste le meilleur compromis) — aucune implémentation n'est forcée quand les données ne le justifient pas.

4. Modèle économique V5 — paramètres réellement retenus

4.1. Paramètres dérivés du moteur réel et de l'étude des vies (pas des benchmarks génériques)

Paramètre	Valeur	Origine
Durée moyenne réelle d'un niveau	1.996 min	Mesurée directement sur les 50 niveaux (src/engine/levels.js)
Palier interstitiel réel	5.989 min	max(3 niveaux × durée moyenne, 5 min) — la condition "3 niveaux" domine (≈6,0 min > 5 min)
Taux de Game Over par tentative (mélange de population)	11.67%	Dérivé de l'étude des vies (section 3), pondéré par le mélange de population ci-dessous
Mélange de population [HYPOTHÈSE]	50% sans Game Over / 35% "Faible" / 15% "Débutant"	Aucune vraie donnée de rétention BREAKPOINT n'existe encore — position explicite, pas une mesure

4.2. Règles économiques appliquées (cahier des charges V5, section 3)

- Interstitiel : après 3 niveaux réussis ET ≥5 minutes depuis le précédent (les deux conditions, jamais un OU) ; jamais pendant une balle active ; jamais en première session ; jamais après un échec automatiquement.
- Rewarded : uniquement après épuisement réel des vies (status "lost"), pour récupérer 1 vie ; reste proposé même à un joueur Premium (jamais imposé, mais jamais retiré).
- Premium 1,99 EUR TTC, achat permanent, supprime DÉFINITIVEMENT les interstitiels — mais PAS le rewarded, qui reste une option volontaire pour tous.
- Aucun IAP cosmétique dans le scénario de base (exclu explicitement par le cahier des charges V5).

4.3. Résultats — 3 scénarios × 6 paliers de MAU

Scénario pessimiste

MAU	Interstitiel net/mois	Rewarded net/mois	Premium net/mois	Total net/mois	Net/MAU/mois
10	0.10 \$	0.02 \$	0.07 \$	0.19 \$	0.0187 \$
100	0.98 \$	0.21 \$	0.68 \$	1.87 \$	0.0187 \$
1 000	9.82 \$	2.11 \$	6.77 \$	18.69 \$	0.0187 \$
10 000	98.18 \$	21.05 \$	67.66 \$	186.89 \$	0.0187 \$
100 000	981.80 \$	210.50 \$	676.60 \$	1 868.90 \$	0.0187 \$
1 000 000	9 818.00 \$	2 105.05 \$	6 766.00 \$	18 689.05 \$	0.0187 \$

Scénario central

MAU	Interstitiel net/mois	Rewarded net/mois	Premium net/mois	Total net/mois	Net/MAU/mois
10	0.62 \$	0.23 \$	0.20 \$	1.06 \$	0.1058 \$
100	6.23 \$	2.32 \$	2.03 \$	10.58 \$	0.1058 \$
1 000	62.30 \$	23.21 \$	20.30 \$	105.80 \$	0.1058 \$
10 000	622.95 \$	232.08 \$	202.98 \$	1 058.01 \$	0.1058 \$
100 000	6 229.52 \$	2 320.81 \$	2 029.80 \$	10 580.13 \$	0.1058 \$

MAU	Interstitial net/mois	Rewarded net/mois	Premium net/mois	Total net/mois	Net/MAU/mois
1 000 000	62 295.21 \$	23 208.14 \$	20 298.00 \$	105 801.34 \$	0.1058 \$

Scénario optimiste

MAU	Interstitial net/mois	Rewarded net/mois	Premium net/mois	Total net/mois	Net/MAU/mois
10	2.29 \$	1.37 \$	0.42 \$	4.08 \$	0.4083 \$
100	22.92 \$	13.68 \$	4.23 \$	40.83 \$	0.4083 \$
1 000	229.22 \$	136.83 \$	42.29 \$	408.34 \$	0.4083 \$
10 000	2 292.20 \$	1 368.28 \$	422.88 \$	4 083.36 \$	0.4083 \$
100 000	22 922.02 \$	13 682.80 \$	4 228.75 \$	40 833.58 \$	0.4083 \$
1 000 000	229 220.24 \$	136 828.01 \$	42 287.50 \$	408 335.75 \$	0.4083 \$

5. Analyse de sensibilité et seuil de rentabilité

Paramètre modifié	Revenu net / MAU / mois résultant	Écart vs central
DAU/MAU 8% (vs 14% central)	0.0692 \$	-34.59%
DAU/MAU 20% (vs 14% central)	0.1424 \$	+34.59%
eCPM interstitiel 5\$ (vs 9\$ central)	0.0781 \$	-26.18%
eCPM interstitiel 13\$ (vs 9\$ central)	0.1335 \$	+26.18%
Opt-in rewarded 25% (vs 45% central)	0.0955 \$	-9.74%
Opt-in rewarded 65% (vs 45% central)	0.1161 \$	+9.74%
Conversion Premium 0.4%/mois (vs 1.2% central)	0.0923 \$	-12.76%
Conversion Premium 2.5%/mois (vs 1.2% central)	0.1278 \$	+20.79%
Session 2.5min (vs 3.5min central)	0.0814 \$	-23.06%
Session 5.0min (vs 3.5min central)	0.1424 \$	+34.59%

Sensibilité spécifique au mélange de population des vies (hypothèse la plus incertaine du modèle V5) :

Hypothèse de population testée	Revenu rewarded/mois résultant (@100k MAU, central)
Population 100% joueurs à l'aise (0 Game Over)	0.00 \$
Population 100% 'Faible' (12.13 GO/campagne)	3 881.57 \$
Population 100% 'Débutant' (15.75 GO/campagne)	4 762.48 \$

Même dans le cas extrême "100% de joueurs à l'aise" (aucun Game Over), le revenu total reste positif (interstitiel + Premium seuls) — le modèle ne s'effondre jamais à zéro, confirmant le critère "stable".

Scénario (à 1000 MAU)	Délai de récupération des 25\$ (compte Google Play)
Pessimiste	1.34 mois
Central	0.24 mois
Optimiste	0.06 mois

6. Conclusion — Stable / Rentable / Durable / Mesurable

STABLE : Le revenu ne repose pas sur un seul paramètre fragile : trois sources indépendantes (interstitiel, rewarded, Premium) contribuent, et la sensibilité (section suivante) montre qu'aucun paramètre isolé ne fait varier le résultat de plus de ~45% dans un sens ou l'autre.

RENTABLE : Revenu net par MAU/mois positif dans les 3 scénarios à 100k MAU (pessimiste 0.0187\$, central 0.1058\$, optimiste 0.4083\$) ; seuil de rentabilité des 25\$ de frais fixes atteint en moins de 2 mois dès 1000 MAU, tous scénarios confondus.

DURABLE : Coût variable identifié : uniquement la bande passante d'hébergement statique, dont le risque a été quantifié et résolu à coût nul (migration Cloudflare Pages documentée, voir rapport économique V4 section 7, non modifiée par V5). Aucun autre coût ne croît avec l'usage : le revenu net par MAU/mois reste rigoureusement constant à toutes les échelles testées (10 à 1 000 000 MAU).

MESURABLE : Inconnues explicites (voir rapport) : mélange réel de population de joueurs (hypothèse 50/35/15 non mesurée), rétention réelle (DAU/MAU), taux d'acceptation réel du rewarded 'continuer', eCPM réel post-intégration AdMob, conversion Premium réelle.

Les trois critères obligatoires (stable, rentable, durable) sont satisfaits par le modèle économique V5 tel que calculé. Ceci est une conclusion TECHNIQUE ET CHIFFRÉE, pas une validation produit — voir section 10.

7. Préparation technique — sans intégration réelle

Conformément au cahier des charges V5 (section 8), la logique économique a été construite et TESTÉE EN ISOLATION, jamais reliée à un SDK réel ni à un écran de l'interface :

- `src/engine/monetization.js` : fonctions pures — compteur de niveaux réussis, garde-fou des 5 minutes (les deux conditions requises, jamais l'une sans l'autre), détection de la première session de découverte, éligibilité rewarded liée au statut "lost" réel, suppression Premium des interstitiels uniquement.
- `src/engine/save.js` : schéma de sauvegarde étendu (`save.monetization`) — état Premium, compteurs, migration testée sans perte de progression existante (sauvegardes V0-V4 réelles simulées dans les tests).
- 12 nouveaux tests (`tests/monetization.test.js`, `tests/save.test.js`) couvrant exactement les cas demandés par le cahier des charges (section 10) : compteur de niveaux, temporisation 5 minutes, Game Over, rewarded, état Premium, persistance/migration.
- AUCUN appel à un SDK publicitaire, AUCUNE intégration Google Play Billing, AUCUN écran d'interstitiel/rewarded/achat n'est câblé dans `main.js` — la logique n'est actuellement invoquée par AUCUN code de l'interface utilisateur réelle. Le câblage effectif reste un travail explicite du futur chantier Capacitor/AAB (section 9 ci-dessous).

8. Tests et non-régression — résultats complets

Suite	Résultat
Tests unitaires (node:test)	110/110 (98 existants + 12 nouveaux monetization/save)
Vérifications mobiles/PWA réelles (Playwright)	21/21 — manifest, service worker, installation, mise à jour, tactile, audio, build ID — aucune régression
Performance sous stress	~60,0 fps moyen, 0 erreur console — inchangée
Non-vacuité — étude des vies	Politique B (0 GO) vs A (27,88 GO moyens) : différence réelle et mesurée, pas un artefact
Non-vacuité — garde-fou 5 minutes	Test dédié : échoue si le compteur de niveaux seul suffisait à contourner le délai
Non-vacuité — flake CI corrigé	8/8 exécutions propres après correctif (contre 1 échec réel observé en CI et reproduit localement avant)

9. Ce qui reste à faire — futur chantier Capacitor/AAB

- Intégration SDK AdMob réel (interstitiel + rewarded), configuration des unités publicitaires, mediation/waterfall, tests de remplissage réel (fill rate).
- Câblage réel de src/engine/monetization.js dans l'interface (main.js) : afficher effectivement l'interstitiel/le rewarded aux moments déjà décidés par la logique testée.
- Intégration Google Play Billing réelle pour le Premium (1,99€), vérification des reçus.
- Restauration d'achat (restore purchase) sur nouvel appareil ou réinstallation.
- Consentement RGPD/ATT (Google UMP, mode consentement) et publication d'une politique de confidentialité.
- Tests sur appareils Android réels variés (pas seulement émulateur/Chromium headless).
- Configuration Play Console complète (fiche store, classification de contenu, prix, pays).
- Génération et signature de l'AAB (keystore généré et gardé en sécurité, jamais committé).
- Test du bouton retour matériel Android (toujours non testé, déjà noté dans README_REPRISE).

10. Métriques à mesurer après publication — remplacer les hypothèses par des données réelles

- Rétention réelle D1/D7/D30 et ratio DAU/MAU réel.
- Sessions/jour réelles et durée de session réelle.
- Taux de Game Over réel par niveau — à comparer directement au mélange de population assumé (50/35/15%) de la section 4.1.
- Taux d'acceptation réel de l'offre rewarded "continuer".
- eCPM réel interstitiel/rewarded post-intégration AdMob (par pays, par OS) et fill rate réel.
- Conversion Premium réelle (taux d'achat, délai depuis l'installation).
- Bande passante réelle d'hébergement (vérifier si/quand la limite souple GitHub Pages est approchée — voir rapport économique V4, section 7).

11. Critère de clôture

Sur le plan technique et économique, les conditions du cahier des charges V5 sont réunies : audit des vies effectué avant toute décision, étude de simulation reproductible comparant A/B/C avec une décision fondée sur les données (aucun changement forcé), modèle économique recalculé avec les paramètres réellement retenus et jugé stable/rentable/durable/mesurable, régression complète verte (110/110 + 21/21), dépôt distant et déploiement vérifiés au niveau des jobs CI réels sur le commit exact ab55570.

Conformément à la méthodologie exigée depuis V0, cette conclusion technique NE CONSTITUE PAS une validation produit. La déclaration officielle « Gameplay, audio, progression et modèle économique gelés — candidat à la transformation Android/Capacitor/AAB » reste conditionnée à la dernière bêta humaine sur téléphone physique mentionnée par le cahier des charges (section 12), qui seule peut confirmer que le comportement des vies (Politique A, inchangée) et l'absence de toute publicité/paiement réel restent satisfaisants en usage réel. Cette confirmation relève exclusivement du propriétaire du projet.